

Assignment 3

Author: Lan Stare

Date: 19. 9. 2025

Question 1

Our starting vertex is 1. That means that it's the first discovered. All the other vertices are undiscovered for now. In the first loop we add all the adjacent vertices of vertex 1 to the stack in decending order. That way number 2 is on top of the stack. We then take the top vertex (2) from the stack. Since 2 doesn't have any adjacent vertices, we don't add anything to the stack. In the next loop, we return to vertex one and take the next vertex from the stack (3) and mark it as explored. We then add 6 and 5 to the stack (6 on top). In the next loop we take 6 from the stack and mark it as explored. Since all of its adjacent vertices are already either discovered or explored, we don't add anything to the stack. In the next loop we take 5 from the stack and mark it as explored. Since 5 doesn't have any adjacent vertices that are unexplored, we don't add anything to the stack. We return to vertex 1, since vertex 4 is the only one left on the stack. In the next loop we take it from the stack and mark it as explored. We have now explored all the vertices and the stack is empty, so the algorithm ends.

Types of edges:

- Tree edges: (1,2), (1,3), (1,4), (3,6), (6,5)
- Backward edges: (6,3), (5,3)
- Forward edges: (3,5)
- Cross edges: (4,3)

Question 2

(a) FALSE

PROOF: If a directed graph G produces exactly one backward edge e during DFS, it means that there is exactly one edge that connects a vertex to one of its ancestors in the DFS tree. However, it doesn't mean, that there is only 1 cycle in the graph. A cycle can be composed of two cross edges. Example: $G = (V, E)$, $V := \{1, 2, 3\}$, $E := \{(1,2), (1,3), (3,1), (2,3), (3,2)\}$. If we apply DFS on graph G starting at vertex 1, then only edge (3,1) is a backward edge. But if we remove only one edge from this graph, we will always have a cycle from 2 to 3 or a cycle from 1 to 3. This means that the graph $G' := (V, E - \{e\})$ is definitely not acyclic no matter which edge e we choose.

(b) FALSE

PROOF: Let's assume that we have a directed graph $G := (V, E)$, $V := \{1, 2, 3, 4\}$, $E := \{(1,2), (1,3), (3,4), (2,4), (4,2)\}$. If we apply DFS starting at vertex 1 and following the same process as in the exercise 1. then we only have tree and cross edges: (1,2), (1,3) and (3,4) are tree edges whereas (2,4) and (4,2) are cross edges. This means that we have not discovered a single backward edge. If we remove the edge (2,4), our graph will become acyclic.

Question 3

```
In [4]: from collections import deque

example_graph2 = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['E'],
    'D': ['E'],
    'E': []}

def min_semesters(graph):
    #For this problem we dont need initialization, we only need to remember what is the Longest path in the graph
    longest_path = {}

    def dfs(u):
        if u in longest_path: #chacking if u is already in longest_path => O(1) because it's a dictionary
            return longest_path[u]
        if not graph[u]: #we add a vertex if it's not already in graph.keys => O(1) because dictionary
            longest_path[u] = 1
            return 1

        #This is our dfs
        max_depth = 0
        for adjacent in graph[u]:
            max_depth = max(max_depth, dfs(adjacent)) # with recursion we can always take the Longest path that we can from u to all other vertices. This is a DFS recursion, which we already k
        longest_path[u] = max_depth + 1
        return longest_path[u]

    return max(dfs(u) for u in graph)

print(min_semesters(example_graph2))
```

4

max() runs DFS once per vertex, but because of longest_path, after the first DFS call on a connected component, all nodes in that component are already computed (saved inside longest_path which has O(1) complexity). So the complexity of this algorithm is still O(n) which is linear.

Question 4

```
In [2]: example_graph2 = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['E'],
    'D': ['E'],
    'E': []}

def has_hamiltonian_path(graph):
    # Initialize
    explored = {u: "UNDISCOVERED" for u in graph}
    topo_order = []

    #Main Loop (this is the way we iplement DFS in python with recursion)
    def dfs(vertex):
        explored[vertex] = "DISCOVERED"
        for w in graph[vertex]:
            if explored[w] == "UNDISCOVERED":
                dfs(w)
        topo_order.append(vertex) #this way we add the vertex at the end of the List when we finish exploring all its adjacent vertices (so we don't have to keep track of the finishing time)

    for v in graph:
        if explored[v] == "UNDISCOVERED":
            dfs(v)

    topo_order.reverse() #we need to reverse the topological order because we added vertices at the end of the List. Using reverse instead of sorting is O(n).

    #Now we need to check the adjacency of each vertex in the top order:
    for i in range(len(topo_order) - 1):
        u, v = topo_order[i], topo_order[i + 1]
        if v not in graph[u]: #if there is no edge between u and v, there is no hamilton path
            return False
    return True

print(has_hamiltonian_path(example_graph2))
```

False